Mobility-Aware Joint Optimization of User Scheduling and Video Delivery with Deep Reinforcement Learning

Yunoh Kim, *Student Member, IEEE,* and Minseok Choi, *Member, IEEE*

***Abstract*—We study high-quality video streaming for high-mobility users under unknown and rapidly varying channels. We propose a two-timescale framework that jointly optimizes (i) queue-driven bitrate and chunk decisions without explicit channel state information (CSI) at a small timescale, and (ii) user-to-roadside unit (RSU) link scheduling via stable matching at a large timescale. The delivery policy is learned by a deep reinforcement learning (DRL) agent whose reward is a frame-based Lyapunov drift-plus-penalty, balancing playback stalls, average quality, and temporal quality fluctuation. Scheduling preferences are built from predicted delivery costs, enabling fair and resource-aware associations under capacity limits. Simulation across diverse SNR and caching conditions show that our CSI-free method achieves consistently low stall rates and competitive average bitrate while mitigating extreme quality switching, outperforming existing baselines.**

***Index Terms*—Deep Reinforcement Learning, Video Delivery, Lyapunov Optimization**

I. Introduction

Online video is now the dominant source of mobile traffic, propelled by over-the-top (OTT) platforms and the proliferation of smart devices. In vehicular settings, where coherence times are short and handovers are frequent, link quality fluctuates quickly, so both access-link scheduling and transport decisions must react fast to maintain smooth playback. These dynamics interact with content locality as roadside units (RSUs) cache only subsets of the catalog, making feasible serving nodes irregular in space and time. As a result, application-layer controllers that treat the wireless path as a quasi-static pipe become brittle; chunk-level adaptive bitrate (ABR) faces nonstationary service rates, while frequent re-associations inflate control overhead.

A conventional response is to inject channel awareness into control. In practice, however, instantaneous or accurately predicted channel state information (CSI) often becomes stale at vehicular speeds. Pilot-based estimation and learning-based prediction alleviate uncertainty but incur error and delay that grow with mobility. This motivates a design that does not rely on explicit CSI, yet remains responsive to queue dynamics and scheduler updates at realistic control cadences. In this regard, we face with several technical challenges for video streaming in vehicular networks.

1) CSI volatility vs. robust delivery

Many edge or mobile edge computing (MEC) frameworks jointly tune bitrate selection, server association, and radio allocation using explicit or predicted CSI; these are effective under slow channels but become fragile when CSI varies within a few chunks, as in fast vehicular motion [1]. deep reinforcement learning (DRL)-based edge streaming typically co-optimizes bitrate and edge selection but still relies on rate/CSI estimates or relatively stable associations [2]. A vehicular two-timescale scheme optimizes bitrate and power over statistical and instantaneous CSI, keeping CSI in the loop [3]. Our design instead avoids instantaneous CSI by driving delivery from queue dynamics.

2) Multi-objective QoE with queue stability

QoE reflects conflicting goals: avoiding stalls, sustaining high average quality, and suppressing temporal fluctuation. Queue-centric formulations couple these objectives to control these objectives to control by regulating playback-buffer dynamics as in [4]. However, many existing ABR/edge systems optimize bitrate or resource use from throughput and average link metrics without analytically stabilizing the playback queue, especially under mobility [5]. We therefore make buffer stability a first-class fast-timescale objective and build delivery decisions directly on the queue.

3) Irregular content availability and dynamic RSU association

Because RSUs cache only subsets and only certain quality levels, where and when chunks can be pushed depends on association and cache locality. Joint edge designs improve QoE by coupling caching and transcoding with radio allocation, but typically presume known/static CSI and do not align scheduler objectives with downstream buffer and QoE dynamics [1], [6].

4) Timescale mismatch and scalability

Recomputing full user-RSU associations at chunk rate is impractical; conversely, forcing delivery to follow slow association cadences loses responsiveness. Two/multi-timescale methods in various domains improve tractability but either keep CSI central or target different architectures, and generally omit buffer stabilization in the fast loop [5], [7], [8]. A vehicular two-timescale live-streaming mechanism similarly optimizes bitrate and bandwidth on statistical CSI and power on instantaneous CSI, leaving queue control out of scope [9]. We require a separation that updates scheduling at a practical cadence while keeping delivery agile and queue-aware.

Our approach. We tackle vehicular streaming by explicitly separating control across timescales and anchoring all fast decisions to the playback buffer rather than to instantaneous CSI.

Y. Kim and M. Choi are with the Department of Electronic Engineering, Kyung Hee University, Yong-In 17104, Korea (e-mail: choims@khu.ac.kr).

Manuscript received April 19, 2005; revised August 26, 2015.

On the fast timescale, delivery actions (bitrate and number of chunks) are chosen CSI-free by a DRL agent whose reward is the per-slot drift-plus-penalty (DPP); this turns "avoid stalls, rasie average quality, suppress oscillation" into a stabilizing signal that the agent can optimize between scheduling rounds. On the slow timescale, RSU-user associations are computed by a deferred-acceptance (DA) matching whose preferences are QoE-aware, so associations directly support downstream playback smoothness and remain robust when CSI is stale. This Lyapunov-guided decomposition yields a tractable per-round pipeline with proven scalability of the matching stage and practical complexity for cost evaluation.

Contributions. Our contributions are summarized as follows.

- We formalize a two-timescale joint optimization for cache-assisted vehicular streaming that decouples slow association from fast delivery, explicitly reflecting mobility, signaling, and computational budgets while accounting for irregular RSU availability due to caching.
- We develop a CSI-free, queue-aware delivery policy: a DPP-rewarded DQN maps buffer dynamics (virtual queue, last quality, distance) to bitrate/chunk actions, improving average bitrate and damping quality fluctuation between scheduling rounds without consuming instantaneous CSI.
- We design QoE-aligned RSU–user scheduling via one-to-many deferred acceptance, using long-horizon DPP costs as preferences; we also provide a tight complexity bound and show how DQN-based cost evaluation keeps per-round overhead practical.
- We present comprehensive evaluations across diverse SNRs, caching distributions, and mobility levels, showing consistent reductions in stalls and bitrate fluctuation with competitive average bitrate versus single-timescale and CSI-dependent baselines; a single Lyapunov weight offers a practical knob for QoE trade-offs.

The remainder of this paper is organized as follows. Section II reviews the related work. Section III introduces the system model, including the video streaming architecture and dynamic queue modeling. Section IV formulates the joint optimization problem and introduces the DRL-based solution for the video delivery subproblem. Section V addresses the scheduling subproblem using a matching algorithm. Section VI provides simulation results to validate the proposed framework. Finally, Section VIII concludes the paper.

II. Related Work

A. Adaptive video streaming

Numerous studies have explored adaptive video streaming for mobile users, yet most overlook the impact of user mobility on dynamic CSI. The authors of [10] jointly optimize bitrate selection, edge transcoding, and client enhancement using average downlink capacity, without modeling channel dynamics. Similarly, [11] focuses on chunk-level quality adaptation without considering mobility or physical-layer information. In [6], Rayleigh fading is incorporated to model wireless channels, yet static CSI is assumed to optimize bitrate and transmission power. The method in [2] performs bitrate allocation under a fixed scheduling assumption, ignoring link rate variation due to mobility.

Other approaches assume partial or full knowledge of instantaneous channel conditions. For example, [12] optimizes encoding bitrate, transmission power, and transcoding based on known channel gain. The system proposed in [13] aims to reduce stall time in LTE/5G networks but does not account for user mobility. The study in [14] analyzes QoE using TCP throughput, yet disregards physical-layer characteristics. The model in [15] adopts a DNN-based QoE predictor and uses network traces for throughput estimation, without modeling CSI fluctuations due to user mobility.

Several works address adaptive streaming tasks in specialized contexts. The framework in [16] applies DRL to solve bitrate selection in UAV-to-UAV delivery, assuming high-mobility but without addressing scheduling or CSI variation. In [17], tile-level bitrate adaptation and viewport-aware tiling are proposed for 360-degree videos, targeting immersive content scenarios. The joint optimization approach in [1] addresses segment caching, transcoding, and resource allocation in MEC networks, yet relies on static channel predictions.

In contrast, our work explicitly addresses the challenge of CSI variability induced by user mobility, and introduces a CSI-free, queue-aware adaptive streaming framework that jointly optimizes scheduling and video delivery across two timescales.

B. QoE-based Bitrate Adaptation

QoE-driven bitrate adaptation has received significant attention, yet many existing works rely on estimated throughput or ignore mobility-induced CSI variations and queue dynamics. A buffer-based iterative bitrate control method was introduced in [18], where playback buffer occupancy guides bitrate decisions without modeling wireless channels or user mobility. A DRL framework assisted by digital twin-based trajectory prediction was proposed in [19], but it utilizes statistically estimated throughput and lacks real-time CSI awareness.

Bitrate adaptation under SINR-driven NOMA channel modeling was addressed in [20], yet user queuing and mobility were not considered. User-level QoE preference was incorporated into bitrate selection in [21], though network dynamics were only indirectly reflected through throughput traces. An adaptive bitrate switching scheme for speech streaming was proposed in [22], which optimizes for packet loss without considering queuing or CSI.

Meta-learning-based approaches have also been proposed. In [23] and [24], meta-RL agents are trained on historical throughput and buffer data for personalized bitrate prediction, but both lack queue modeling and CSI considerations. An online learning method was studied in [25] using 4G LTE traces and playback buffer constraints, yet it does not account for physical-layer channel variation.

Several works focused on edge-aware or CSI-aware streaming optimization. Bitrate adaptation under edge caching constraints was studied in [26], where average bitrate is maximized without QoE modeling. A joint optimization of RSMA parameters and bitrate was proposed in [27], relying on

fully observed CSI but omitting queue dynamics. Similarly, beamforming-assisted video delivery in cell-free massive MIMO was addressed in [28], but without buffering or mobility modeling.

Adaptive video streaming in massive MIMO networks using RL was studied in [29], assuming perfect CSI and buffer risk modeling, yet without incorporating mobility. A two-timescale framework for vehicular networks was introduced in [3], optimizing bitrate and power allocation using statistical and instantaneous CSI, but excluding queue modeling. Additional studies have explored specialized use cases, such as viewport-adaptive 360-degree streaming [30], coordinated audio-video bitrate control [31], and joint bitrate and FEC rate adaptation for real-time video streaming [32], yet none comprehensively address CSI uncertainty and queuing under mobility.

In contrast, our work performs CSI-free, queue-aware bitrate adaptation while accounting for user mobility, and integrates this into a two-timescale joint optimization framework for scheduling and delivery.

C. Wireless Link Scheduling

Wireless link scheduling has been extensively studied in various network contexts, yet most existing methods assume static CSI and do not consider user queuing systems or mobility-induced dynamics. An unsupervised clustering approach for MIMO scheduling was proposed in [33], relying on CSI similarity without queue or mobility considerations. A data-importance-aware scheduler was introduced in [34] to reduce training overhead in edge learning, but scheduling decisions are made without mobility or queuing dynamics.

Joint scheduling and power control were explored in SWIPT-aided federated learning using DRL in [35], yet the approach requires full CSI and ignores user queue states. Integrated terrestrial-satellite scheduling and hybrid beamforming were optimized in [36], under the assumption of perfect CSI and static user locations. Similarly, linear beamforming and scheduling in MISO downlink systems were jointly designed in [37], but without modeling queuing or mobility.

A deep unfolding method for scheduling, clustering, and beamforming was studied in [38], where perfect or partial CSI was used, and queue states were ignored. A learning-based scheduler for RIS-assisted downlink was proposed in [39], but only static channels and fixed user positions were considered. Queuing was modeled in [40] for XR services with strict latency constraints, but mobility was not addressed and perfect CSI was assumed.

In vehicular networks, joint task scheduling and computation offloading were studied in [41] and [42], both of which incorporate mobility prediction or VES control but do not explicitly model queuing. A backhaul-constrained user association and power allocation method using matching theory was proposed in [43], assuming fully known CSI and static user locations.

Matching-based approaches have been explored for distributed scheduling. Stable matching for D2D resource allocation with energy-harvesting relays was developed in [44], assuming known SINR but ignoring queuing and mobility. A two-level matching framework for cognitive femtocell networks was introduced in [45], jointly optimizing user association and subchannel allocation without queue awareness. Deferred acceptance-based matching was also applied in ultra-reliable low-latency systems in [46], where CSI is estimated and treated as known, but queue dynamics are not modeled. Scheduling in multi-cell NOMA networks was formulated as a student-project allocation game in [47], where fairness and SINR constraints are optimized via stable matching, again under static CSI and no queuing assumptions.

In contrast, our work explicitly accounts for queue dynamics and mobility-induced CSI uncertainty, and performs link scheduling under resource constraints using matching theory, enabling efficient and fair delivery in realistic wireless caching networks.

III. System Model

Section III outlines the overall system architecture, introduces the two-timescale control structure for delivery and scheduling, and presents the queue dynamics including virtual queue modeling for playback stability.

A. Vehicular Network Model

We study a video streaming framework operating in a multi-user vehicular network with multiple content sources (e.g., RSU), as depicted in Fig. 1. We consider a mobility-aware video streaming scenario in which a total of N vehicles traverse a monitored road segment of length d_m and width d_w. Along both sides of the road, K RSUs are uniformly deployed at regular intervals of d_g, forming a grid layout. Each vehicle $n \in \mathcal{N} \triangleq \{1, \cdots, N\}$ moves with a constant speed $v_n \in [V_{min}, V_{max}]$ and updates its position at each discrete time slot $t \in \{0, 1, \cdots\}$. New vehicles probabilistically appear on the road every T slots according to a Poisson arrival process with rate λ. The distance between vehicle n and RSU $k \in \mathcal{K} \triangleq \{1, \cdots, K\}$ at time t is denoted by $d_{kn}(t)$. Under the mobility-aware assumption, we assume that the vehicle speed v_n is known and $d_{kn}(t)$ can be updated at each time slot. Accordingly, we define the set of schedulable RSUs for user n at time t as $\mathcal{K}_n(t) \triangleq \{k \in \mathcal{K} \mid d_{kn}(t) \leq R_s\}$, which includes all RSUs located within the searching range R_s of user n. Each user scans this range to identify feasible RSUs for potential association at the beginning of a scheduling round. Similarly, the set of schedulable users for RSU k at time t is given by $\mathcal{N}_k(t) \triangleq \{n \in \mathcal{N} \mid d_{kn}(t) \leq R_s\}$, capturing all users within the RSU's searching range. These sets are dynamically updated at each slot according to user mobility and serve as the basis for scheduling decisions.

It is important to clarify that the searching range R_s is applied only when forming the candidate set of RSUs at the beginning of an association round. Once a user is associated with a particular RSU, this link remains valid for the entire round, even if the user's movement later causes the distance to exceed R_s. In other words, the searching range functions as a criterion for initiating or updating associations, rather than a strict connectivity limit during transmission. As long as the

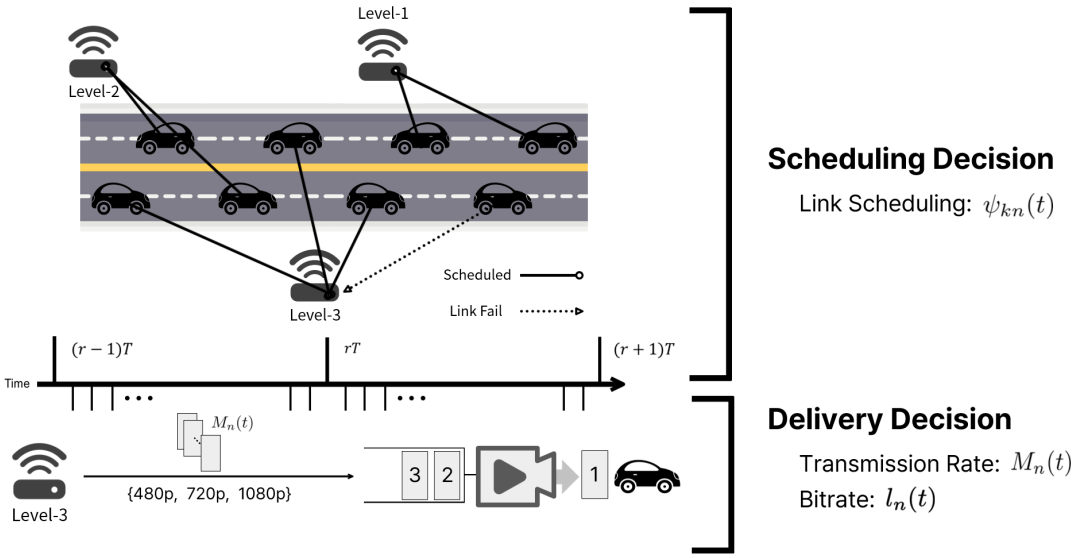

Fig. 1: Two-timescale link scheduling and video delivery

established link continues to satisfy the channel capacity constraints, which will be detailed in Section III-B, the ongoing data delivery is maintained without triggering immediate re-association.

In practice, each RSU can simultaneously serve only a limited number of users, while each user is typically connected to a single RSU for video reception. In this regard, to capture the most competitive scenario, we assume that all vehicular users request the same video content which can be encoded into the versions having L different quality levels and that only RSUs caching this content participate in delivery. Nevertheless, the proposed framework can be readily extended to the more general case where users request heterogeneous content and RSUs cache different items. In such situations, each user would independently form a candidate set consisting only of RSUs storing its requested content and send a scheduling request to the RSU with the lowest expected delivery cost. Each RSU, in turn, would simply apply the DA-based matching algorithm to the set of requesting users, irrespective of whether their requested contents differ.

We suppose that K RSUs cache the desired item, but at varying quality levels. Specifically, RSU k stores the content at quality level q_k, which is randomly selected according to a predefined caching probability distribution $\mathbf{p} = [p_1, \cdots, p_L]$, where p_i represents the probability that a node caches the content at quality level i. Once associated with an RSU k with caching quality level q_k, user n selects the quality level $l_n(t) \in \mathcal{Q}_k \triangleq \{1, \cdots, q_k\}$, constrained by the RSU's cached content quality.

B. Two-Timescale Procedure for Video Delivery

To support high-quality video streaming in mobile environments, we adopt a two-timescale framework that decouples the scheduling of access nodes and the delivery of video chunks. This separation reflects both the temporal disparity and the control-plane cost of the two tasks. In particular, association/scheduling operates on a slow timescale because establishing or changing a user-RSU link requires multiple signaling steps, e.g., neighbor discovery, random-access and link establishment, authentication/authorization, QoS configuration and resource admission, which introduce non-negligible latency and overhead. Executing this end-to-end procedure every slot would be inefficient; instead, we update associations only once per scheduling round and maintain them throughout the round to amortize signaling cost and ensure stability. By contrast, once a link is active, the RSU can adjust chunk bitrate and transmission quantity on the fast timescale using local state with minimal control overhead. Consequently, scheduling is resolved on a long horizon for robustness and efficiency, while chunk delivery adapts every slot for responsiveness to rapid channel and buffer dynamics.

Therefore, on a large time scale, each user selects a source node, that is, RSU, based on spatial proximity, current queue status introduced in Section III-C, and the expected cumulative QoE metrics discussed in Section IV over the duration of delivery. While association requests are triggered independently by users, the actual scheduling decisions are coordinated system-wide and resolved synchronously at the beginning of each scheduling round. Once the scheduling round r is triggered at time t_n^r for user n, the association remains fixed over the interval $\mathcal{T}_n^r = \{t_n^r, t_n^r + 1, \cdots, t_n^r + T - 1\}$, during which the user is exclusively served by the selected RSU. Let $\psi_{kn}(t)$ denote the scheduling indicator, where $\psi_{kn}(t) \in \{0, 1\}$ indicates that user n is associated with node k at time t. Then, for user n, $\boldsymbol{\psi}_n(t) = [\psi_{1n}(t), \psi_{2n}(t), \cdots, \psi_{Kn}(t)]$ is fixed for the interval $t \in \mathcal{T}_n^r$ given round r, and $\sum_{k \in \mathcal{K}} \psi_{kn}(t) \in \{0, 1\}$ because each user can be associated with a single RSU at most but sometimes it fails to find the source node within the searching range. Each RSU can serve up to J users concurrently in a round, and the final associations are determined according to the matching framework described in Section V.

Each video is encoded into a sequence of equal-duration

chunks, where each chunk can be transmitted and decoded independently. Users sequentially receive and buffer these chunks during the connection interval with a source node. Given the association between user n and RSU k, RSU k determines the quality level $l_n(t)$ and the number of chunks $M_n(t) \in \{0, 1, \cdots\}$ to transmit to user n at each time slot $t \in \mathcal{T}_n^r$. The quality selection is constrained by the set of available quality levels $\mathcal{Q}_k$ stored at RSU k, and let $S(l_n(t))$ denote the size of a chunk with quality level $l_n(t)$. Then, given the wireless channel capacity $C_{kn}(t)$, the delivery must satisfy the per-slot constraint:

$$M_n(t) \cdot S(l_n(t)) \leq t_0 \cdot C_{kn}(t) \cdot \psi_{kn}(t), \tag{1}$$

where t_0 is the duration of a time slot, and W is the bandwidth. The system operates under orthogonal access, such that each user transmits over a dedicated channel with bandwidth W, avoiding intra-node interference.

The channel capacity $C_{kn}(t)$ follows a Rayleigh fading model, incorporating path loss, shadowing, and fast fading. Specifically, the channel gain $h_{kn}(t)$ is given by

$$h_{kn}(t) = \sqrt{D_{kn}(t) \cdot w(t)} \cdot u(t), \tag{2}$$

where $D_{kn}(t) = d_{kn}(t)^{-\beta}$ represents path loss, $w(t) \sim \mathcal{N}(0, \sigma^2)$ models shadowing, and $u(t) \sim \mathcal{CN}(0, 1)$ captures fast fading. As a result, the channel capacity $C_{kn}(t)$ can vary significantly over time due to mobility-induced CSI dynamics. The corresponding channel capacity is then expressed as:

$$C_{kn}(t) = W \cdot \log_2 \left(1 + \frac{\Gamma |h_{kn}(t)|^2}{\Upsilon + 1}\right), \tag{3}$$

where Γ is the transmit SNR, and Υ denotes the interference-to-noise ratio (INR).

In summary, the proposed framework operates on two distinct timescales, as illustrated in Fig. 1. At every T slots, the slow-timescale controller determines the scheduling indicators $\psi_{kn}(t)$, thereby finalizing the user-RSU associations for the upcoming round. Once the associations are fixed, each active link adaptively decides the chunk quantity $M_n(t)$ and quality $l_n(t)$ on a per-slot basis to respond to queue dynamics and instantaneous channel conditions. Importantly, the scheduling phase must anticipate the stochastic evolution of delivery actions: once a link is activated, the ensuing sequence of $\{M_n(t), l_n(t)\}$ is probabilistically determined by buffer status, user mobility, and fading effects, which jointly contribute to the system cost. Hence, each user evaluates its candidate RSUs by predicting the expected cumulative cost over the scheduling horizon and sends a request to the one that minimizes this metric, while each RSU, upon receiving more than J simultaneous requests, revolves contention by admitting the subset of users that achieve the lowest expected cost under the matching rule. By explicitly coupling the long-term stability of scheduling decisions with the short-term responsiveness of video delivery, the two-timescale structure depicted in Fig. 1 provides a scalable and responsive solution for consistent video streaming under real-world mobility and channel dynamics.

C. Queue Dynamics for Delivery Adaptation

Each user maintains a playback buffer that stores received but unplayed video chunks. The queue length $Q_n(t)$ evolves according to

$$Q_n(t+1) = \max\{Q_n(t) - b, 0\} + M_n(t), \ \forall n \in \mathcal{N} \tag{4}$$

where $Q_n(0) = 0$ and b denotes the number of chunks consumed per slot by the video player. Here, we assume that the display time required to play b chunks is equal to the coherence time t_0 of the video delivery process. A playback stall occurs if the buffer becomes empty, i.e., $Q_n(t) = 0$, indicating that the user cannot continue playback in that slot.

The dynamics of queuing in (4) capture the trade-off between suppressing playback stalls and improving video quality. The delivery policy must balance two opposing goals: increasing $M_n(t)$ to prevent buffer underflow and selecting a higher $l_n(t)$ to enhance video quality. Due to the link capacity constraint, increasing one may reduce the other, especially under poor channel conditions. Thus, the queue acts as both a buffer and a feedback mechanism, enabling adaptive bitrate control without explicit CSI.

To avoid playback stalls, it is essential to maintain a non-empty playback queue at the user side. However, enforcing strict queue constraints at every time slot is impractical due to the stochastic and time-varying nature of wireless environments. Instead, we aim to pursue that the long-term time-averaged queue backlog should be larger than any nonnegative constant c:

$$\lim_{R \to \infty} \frac{1}{N} \sum_{n=1}^{N} \frac{1}{RT} \sum_{r=1}^{R} \sum_{t \in \mathcal{T}_n^r} \mathbb{E}[Q_n(t)] > c, \tag{5}$$

where R is the number of total scheduling rounds. According to [48], [49], we define an auxiliary virtual queue $Z_n(t) = \tilde{Q} - Q_n(t)$ for all $n \in \mathcal{N}$, where $\tilde{Q}$ is a sufficiently large constant, and pursue the stability of $Z_n(t)$ to satisfy (5). This virtual queue evolves according to

$$Z_n(t+1) = \min\{Z_n(t) + b, \tilde{Q}\} - M_n(t), \ \forall n \in \mathcal{N}, \tag{6}$$

where $Z(0) = \tilde{Q}$. Intuitively, if $Z_n(t)$ remains strongly stable, i.e., the long-term expected length of $Z_n(t)$ is upper bounded to a finite value, then $Q_n(t)$ could remain bounded away from zero with high probability, thereby preventing the buffer from being frequently depleted. This approach enables us to ensure that the long-term time-averaged backlog of $Q_n(t)$ is positive and finite, which in turn guarantees that video chunks are persistently accumulated in the playback buffer. In other words, stabilizing the auxiliary process $Z_n(t)$ suppresses playback stall events, and thus provides a tractable analytical framework to design delivery policies that achieve both queue stability and smooth video streaming performance.

IV. Joint Optimization Problem for User Scheduling and Video Delivery

A. Problem Formulation

To ensure long-term streaming quality, our objective is to maximize the average quality of received video chunks among

all users during R rounds of scheduling which is derived as

$$\frac{1}{N}\sum_{n=1}^{N}\frac{1}{RT}\sum_{r=1}^{R}\sum_{t\in\mathcal{T}_n^r}\left(\frac{\mathcal{P}(l_n(t))M_n(t)}{\sum_{r'=1}^{R}\sum_{t'\in\mathcal{T}_n^{r'}}M_n(t')}\right), \quad (7)$$

where $\mathcal{P}(l_n(t))$ denotes the quality measure of the chunk encoded at level $l_n(t)$ (e.g., PSNR and SSIM). However, it is difficult to anticipate the total number of received chunks over a long-term horizon, i.e., $\sum_{r'=1}^{R}\sum_{t'\in\mathcal{T}_n^{r'}}M_n(t')$ because we adaptively determine $M_n(t)$ every slot. Therefore, we instead minimize the time-averaged degradation of quality measures of received chunks, which is expressed by

$$\frac{1}{N}\sum_{n=1}^{N}\frac{1}{RT}\sum_{r=1}^{R}\sum_{t\in\mathcal{T}_n^r}\left(\overline{\mathcal{P}}-\mathcal{P}(l_n(t))\right)\cdot M_n(t), \quad (8)$$

where $\overline{\mathcal{P}}$ is the maximum quality measure. The simplest approach to minimize (8) is to never transmit chunk, i.e., $M_n(t)=0$ for all t and n, which is identical to maximization of (7) by making the denominator zero; however, our long-term time-averaged queue constraint in (5), which is equivalent to stability of $Z_n(t)$, avoids this approach.

In addition to maximizing the average video quality and ensuring virtual queue stability to avoid playback stalls, it is also critical to suppress excessive quality fluctuations, since abrupt variations in bitrate significantly degrade the user's QoE even when the long-term average quality is high. To capture these requirements, we formulate the following two-timescale joint optimization problem for scheduling and video delivery:

$$\min_{\boldsymbol{\psi},\mathbf{l},\mathbf{M}}\lim_{R\to\infty}\mathbb{E}\Bigg[\frac{1}{N}\sum_{n=1}^{N}\frac{1}{RT}\sum_{t\in\mathcal{T}_n}\Bigg\{\left(\overline{\mathcal{P}}-\mathcal{P}(l_n(t))\right)\cdot M_n(t) + \varphi\cdot|l_n(t)-\tilde{l}_n(t)|\Bigg\}\Bigg] \quad (9)$$

$$\text{s.t.}\ \lim_{R\to\infty}\frac{1}{N}\sum_{n=1}^{N}\frac{1}{RT}\sum_{t\in\mathcal{T}_n}\mathbb{E}[Z_n(t)]<\infty, \quad (10)$$

$$M_n(t)S(l_n(t))\le t_0C_{kn}(t)\psi_{kn}(t),\ \forall k\in\mathcal{K},\ \forall n\in\mathcal{N} \quad (11)$$

$$\psi_{kn}(t)\in\{0,1\},\quad \forall k\in\mathcal{K},\ \forall n\in\mathcal{N} \quad (12)$$

$$\sum_{k=1}^{K}\psi_{kn}(t)\le 1,\quad \forall n\in\mathcal{N} \quad (13)$$

$$\sum_{n=1}^{N}\psi_{kn}(t)\le J,\quad \forall k\in\mathcal{K} \quad (14)$$

$$l_n(t)\in\mathcal{Q}_k,\quad \forall\{n,k\}\text{ with }\psi_{kn}(t)=1, \quad (15)$$

where $\tilde{l}_n(t)$ represents the quality level of the most recently delivered chunk prior to time slot t, and $\varphi\ge 0$ is the weighting factor for the penalty for quality fluctuations. Here, we note that the agent could deliberately choose not to transmit any chunk in a given slot, e.g., when the channel is poor or when sufficient chunks are accumulated in the queue. As a result, the fluctuation term $|l_n(t)-\tilde{l}_n(t)|$ should not be interpreted as the difference between qualities selected in two consecutive slots. Instead, it measure the change in quality between the newly delivered chunk at slot t and the last chunk that actually arrived at the user buffer. Since the playback process consumes chunks in their order of arrival, this formulation faithfully captures the true fluctuation in perceived video quality, regardless of whether chunks are delivered in consecutive slots or with gaps. The optimal solution is given by $\{\boldsymbol{\psi}^\star,\mathbf{l}^\star,\mathbf{M}^\star\}$, where the decision variables are defined as $\boldsymbol{\psi}=[\boldsymbol{\psi}_{kn},\forall k,n]$, $\mathbf{l}=[\mathbf{l}_1,\mathbf{l}_2,\cdots,\mathbf{l}_N]$, and $\mathbf{M}=[\mathbf{M}_1,\mathbf{M}_2,\cdots,\mathbf{M}_N]$. Each element spans RT time slots, where $\boldsymbol{\psi}_{kn}=[\psi_{kn}(0),\cdots,\psi_{kn}(RT-1)]$, $\mathbf{l}_n=[l_n(0),\cdots,l_n(RT-1)]$, and $\mathbf{M}_n=[M_n(0),\cdots,M_n(RT-1)]$.

The objective function in (9) minimizes the average weighted sum of quality degradation and inter-slot quality fluctuation. The first term penalizes the deviation of the delivered chunks from the maximum quality level $\overline{\mathcal{P}}$, thereby incentivizing higher-quality streaming, while the second term, weighted by φ, discourages abrupt quality switches. Notably, the weighting structure links the chunk quantity $M_n(t)$ with quality decisions: when a higher-quality level $l_n(t)$ is selected, the penalty becomes smaller, which effectively reduces the cost of transmitting more chunks. Conversely, when quality is low, the penalty is larger, making additional chunk transmissions more costly. As a result, the objective function naturally aligns with the virtual queue stability requirement in (10): it encourages the system to deliver more chunks precisely when quality is high, thereby both maintaining buffer stability and improving average playback quality.

The stability constraint in (10) further ensures that the expected backlog of the virtual queues $\mathbf{Z}(t)=[Z_1(t),\cdots,Z_N(t)]$ remains finite. This strong stability condition implies that the actual playback queues $Q_n(t)$ seldom empty, directly suppressing playback stalls. Together with the weighting mechanismo the objective in (9), the system balances three competing factors: (i) pushing up the average video quality, (ii) maintaining sufficient buffering to avoid playback interruption, and (iii) smoothing out quality variations across slots. The remaining constraints impose feasibility and fairness: (11) enforces the channel capacity limitation, (12)–(14) guarantee valid user–RSU associations under per-node capacity J, and (15) restricts quality selection to cached levels available at each RSU.

B. Lyapunov-based Problem Decomposition

To solve the problem defined in (9)–(15), we first address the queue stability constraint (10) by employing Lyapunov optimization theory [50]. We define the Lyapunov function, using indices r and i for slow and fast timescales respectively, as

$$L(r,i)\triangleq\frac{1}{2N}\sum_{n=1}^{N}Z_n(t_n^{(r,i)})^2, \quad (16)$$

where $t_n^{(r,i)}=t_n^r+i$. The queue stability is pursued by minimizing the expected round-based Lyapunov drift, defined as

$$\Delta(r)\triangleq\mathbb{E}[L(r+1,0)-L(r,0)|\mathbf{Z}(r)], \quad (17)$$

where $\mathbf{Z}(r) \triangleq [Z_n(t_n^r) : \forall n \in \mathcal{N}]$. The upper bound of the one-slot drift is given by

$$\begin{aligned} &L(r,i+1) - L(r,i) \\ &\leq \frac{1}{2N}\sum_{n=1}^{N}\left\{Z_n(t_n^{(r,i)}+1)^2 - Z_n(t_n^{(r,i)})^2\right\} \\ &\leq \frac{1}{2N}\sum_{n=1}^{N}\Bigg\{\min\Big\{Z_n(t_n^{(r,i)}) - M_n(t_n^{(r,i)}) + b, \\ &\qquad \tilde{Q} - M_n(t_n^{(r,i)})\Big\}^2 - Z_n(t_n^{(r,i)})\Bigg\} \\ &\leq \frac{1}{2N}\sum_{n=1}^{N}\left\{(Z_n(t_n^{(r,i)}) - M_n(t_n^{(r,i)}) + b)^2 - Z_n(t_n^{(r,i)})^2\right\}, \end{aligned} \tag{18}$$

which holds due to the virtual queue update rule and bounded drift term.

By summing over $i \in \{0, \cdots, T-1\}$, the round-based Lyapunov drift is upper-bounded as

$$\begin{aligned} &L(r+1,0) - L(r,0) \leq \\ &\frac{1}{N}\sum_{n=1}^{N}\sum_{t\in\mathcal{T}_n^r}\left\{Z_n(t)(b - M_n(t)) + \frac{1}{2}(b - M_n(t))^2\Big|\mathbf{Z}(r)\right\}. \end{aligned} \tag{19}$$

Accordingly, the drift-plus-penalty (DPP) objective is upper-bounded by

$$\begin{aligned} f(\boldsymbol{\psi}, \mathbf{l}, \mathbf{M}, \mathbf{Z}(r)) = \mathbb{E}\Bigg[&\frac{1}{N}\sum_{n=1}^{N}\sum_{t\in\mathcal{T}_n^r}\Bigg\{Z_n(t)(b - M_n(t)) \\ &+ V\cdot\left(\overline{\mathcal{P}} - \mathcal{P}(l_n(t))\cdot M_n(t) + \varphi|l_n(t) - \tilde{l}_n(t)|\right) \\ &+ \frac{1}{2}(b - M_n(t))^2\Big|\mathbf{Z}(r)\Bigg\}\Bigg]. \end{aligned} \tag{20}$$

To simplify the decomposition of the joint optimization problem, we approximate the cost of the DPP per user by excluding the quadratic term $\frac{1}{2}(b - M_n(t))^2$ with the assumption that $M_n(t)$ is upper bound to a finite constant. Then, the per-user cost function is defined as

$$\begin{aligned} f_n(l_n(t), M_n(t), \boldsymbol{\psi}_n(t)) &= Z_n(t)(b - M_n(t)) \\ &+ V\cdot\left(\overline{\mathcal{P}} - \mathcal{P}(l_n(t))M_n(t) + \varphi|l_n(t) - \tilde{l}_n(t)|\right). \end{aligned} \tag{21}$$

Thus, the overall DPP can be equivalently approximated as

$$\begin{aligned} &f(\boldsymbol{\psi}, \mathbf{l}, \mathbf{M}, \mathbf{Z}(r)) \\ &\approx \mathbb{E}\left[\frac{1}{N}\sum_{n=1}^{N}\sum_{t\in\mathcal{T}_n^r} f_n(l_n(t), M_n(t), \boldsymbol{\psi}_n(t))\Big|\mathbf{Z}(r)\right]. \end{aligned} \tag{22}$$

Due to the difficulty of computing the expected value in the round-based DPP objective in (20), we adopt an opportunistic minimization approach and solve the instantaneous DPP problem at each round r [50]. The per-round minimization problem is formulated as:

$$\begin{aligned} \min_{\boldsymbol{\psi}_n^r, \mathbf{l}_n^r, \mathbf{M}_n^r} \ &\frac{1}{N}\sum_{n=1}^{N}\sum_{t\in\mathcal{T}_n^r} f_n(l_n(t), M_n(t), \boldsymbol{\psi}_n(t)) \\ \text{s.t.} \ &(11) - (15), \end{aligned} \tag{23}$$

where $\boldsymbol{\psi}_n^r = [\psi_{kn}(t) : \forall k \in \mathcal{K}_n(t), \forall t \in \mathcal{T}_n^r]$, $\mathbf{l}_n^r = [l_n(t) : t \in \mathcal{T}_n^r]$, and $\mathbf{M}_n^r = [M_n(t) : t \in \mathcal{T}_n^r]$. Here, the scheduling decision of user n, i.e., $\psi_{kn}(t)$ for all $k \in \mathcal{K}_n(t)$, remains fixed over the round, while $l_n(t)$ and $M_n(t)$ could vary across slots.

Given a candidate scheduling decision $\boldsymbol{\psi}' = [\boldsymbol{\psi}'_n : \forall n \in \mathcal{N}]$ that satisfies (12)–(14), the video delivery subproblem becomes separable among users due to the orthogonality of activated links, which is given by

$$\begin{aligned} \min_{\mathbf{l}_n^r, \mathbf{M}_n^r} \ &\sum_{t\in\mathcal{T}_n^r} f_n(l_n(t), M_n(t), \boldsymbol{\psi}'_n) \\ \text{s.t.} \ &(11), (15). \end{aligned} \tag{24}$$

Let the optimal delivery decisions under $\boldsymbol{\psi}'_n$ be denoted by $\{\mathbf{l}_n^{r*}(\boldsymbol{\psi}'), \mathbf{M}_n^{r*}(\boldsymbol{\psi}')\}$. Based on these, the link scheduling subproblem is expressed as

$$\begin{aligned} \min_{\boldsymbol{\psi}^r} \ &\frac{1}{N}\sum_{n=1}^{N}\sum_{t\in\mathcal{T}_n^r} f_n(\mathbf{l}_n^{r*}(\boldsymbol{\psi}^r), \mathbf{M}_n^{r*}(\boldsymbol{\psi}^r), \boldsymbol{\psi}^r) \\ \text{s.t.} \ &(12), (13), (14). \end{aligned} \tag{25}$$

Having decomposed the original joint problem into the video delivery subproblem in (24) and the link scheduling subproblem in (25), we now describe how these two components are coordinated to solve the overall optimization. The key idea is that solving the scheduling problem requires, as an intermediate step, knowledge of the optimal delivery actions for each candidate association. Importantly, given a scheduling policy $\boldsymbol{\psi}^r$, the optimal delivery decisions $\{\mathbf{l}_n^{r*}(\boldsymbol{\psi}^r), \mathbf{M}_n^{r*}(\boldsymbol{\psi}^r)\}$ depend only on user n's scheduling of $\boldsymbol{\psi}_n^r$, not on the scheduling decisions of other users. Hence, each user can independently evaluate its delivery cost $f_n(.)$ by solving the video delivery subproblem (24) for all RSU candidates within its searching range. This results in a set of cost values, one for each potential user-RSU association, and will be further elaborated in Section IV-C.

Based on these computed costs, each user sequentially sends a scheduling request to the RSU that minimizes its own expected cost. Since each RSU can serve at most J users, conflicts may arise when more than J users simultaneously request association with the same RSU. In such cases, the RSU invokes a stable matching process that selects up to J users so as to minimize the aggregated system cost in (25), while rejecting the remaining requests. Rejected users then proceed to their next preferred RSU, repeating the process until a stable allocation is reached. This process will be explained more in detail in Section V.

Through this two-stage procedure, the video delivery subproblem provides per-user cost evaluations under all feasible associations, and the link scheduling subproblem coordinates these outcomes across users and RSUs under capacity constraints. Together, they yield an implementable policy that

minimizes the DPP objective while ensuring fairness and stability in the multi-user and multi-RSU environment.

C. Reinforcement Learning-based Video Delivery

To address the video delivery subproblem in (24) under unknown and time-varying channel conditions, we adopt a model-free DRL approach. Since both the quality level $l_n(t)$ and the number of chunks $M_n(t)$ take values from finite discrete sets, a Deep Q-Network (DQN) is employed to learn a delivery policy over the horizon $t \in \mathcal{T}_n^r$ that minimizes the cumulative queue-aware cost, i.e., $\sum_{t\in\mathcal{T}_n^r} f_n(l_n(t), M_n(t), \boldsymbol{\psi}'_n)$, under a fixed association, $\psi_{kn}(t) = 1$ and $\psi_{in}(t) = 0$ for all $i \in \mathcal{K}_n(t) \setminus \{k\}$.

We model the delivery task for each user-RSU pair (n, k) as a Markov decision process (MDP) $\mathcal{M} = \{\mathcal{S}, \mathcal{A}, P, R\}$, where $\mathcal{S}$ and $\mathcal{A}$ denote the state and action spaces, respectively; P is the state transition probability, and R represents the reward. The state for a user-RSU pair (n, k) at time t is defined as $s_t^{(n,k)} = \{Z_n(t), \tilde{l}_n(t), d_{kn}(t)\}$, where $Z_n(t)$ is the virtual queue length, $l_n(\tau_t)$ is the quality level of the most recently delivered chunk prior to slot t, and $d_{kn}(t)$ is the distance between user n and RSU k. The action is given by $a_t^{(n,k)} = \{M_n(t), l_n(t)\}$. The system follows a deterministic transition model, where $s_{t+1}^{(n,k)}$ is uniquely determined by $s_t^{(n,k)}$ and $a_t^{(n,k)}$, based on user mobility and delivery decisions.

In order to resolve the video delivery problem in (24), the reward of each user-RSU pair (n, k) at slot t is formulated as the negative of the per-slot objective, which is given by

$$r_t^{(n,k)} = R(s_t^{(n,k)}, a_t^{(n,k)}) = -f_n(l_n(t), M_n(t)), \tag{26}$$

where we omitted the dependency of $f_n(.)$ on $\boldsymbol{\psi}'_n$ because $\psi_{kn}(t) = 1$ is already assumed for the given pair (n, k). This reward design aligns the learning objective with the goal of minimizing playback stalls and quality fluctuation, enabling the agent to learn effective delivery strategies even without explicit CSI.

Although the environment evolves deterministically with respect to the current action, predicting the sequence of optimal actions over $t \in \mathcal{T}_n^r$ is challenging due to the absence of future CSI. To address this, we design the DQN-based video delivery to adapt to highly dynamic channel conditions while maximizing the cumulative reward over the decision horizon. The DQN agent stores its experience tuples $x_t = (s_t, a_t, r_t, s_{t+1})$ in a replay memory $\mathcal{D}$, from which training samples are drawn. Once sufficient experience tuples have been accumulated in the replay buffer $\mathcal{D}_t$ at time step t, the DQN parameterized by θ is trained to approximate the optimal action-value function $Q^*(s, a)$. To stabilize learning, a separate target network parameterized by ϕ is maintained, and the target Q-value is computed as

$$\hat{Q}_\phi(s_t, a_t) = r_t + \gamma \cdot \max_{a_{t-1}} Q_\phi(s_{t-1}, a_{t-1}), \tag{27}$$

where $\gamma \in (0, 1)$ is the discount factor. The DQN network θ is updated by minimizing the loss

$$\mathcal{L}(\theta) = \left(\hat{Q}_\phi(s_t, a_t) - Q_\theta(s_t, a_t)\right)^2, \tag{28}$$

via stochastic gradient descent over mini-batches sampled from $\mathcal{D}$. The target network ϕ is synchronized with the main network θ every T_{target} steps to enhance training stability.

During operation, each RSU k executes the trained DQN to evaluate scheduling requests from users n within its coverage. For each candidate user, the RSU simulates the delivery process over $\mathcal{T}_n^r$, assuming $\psi_{kn}(t) = 1$, and uses the DQN to infer $\{l_n(t), M_n(t)\}$ at each slot. The resulting cumulative cost serves as the matching metric defined in (24). Although this evaluation is performed per user, the computational burden remains manageable, as each user typically considers only a small set of nearby RSUs. The DQN is trained online throughout the episode by periodically aggregating experience tuples from multiple vehicles every T slots and updating the network parameters in mini-batches. This enables adaptive learning to dynamic mobility and channel variations while maintaining computational efficiency.

The DRL-based delivery decision process under a fixed scheduling policy $\psi_{kn}(t) = 1$ is summarized in **Algorithm 1**, where the DQN sequentially selects the chunk quality and quantity to minimize the per-user DPP objective over the decision horizon.

Algorithm 1 Training procedure of DQN agent

Notation

D_m : replay memory size.

D_s : minimum buffer size to start training.

DQN agent Q_θ
Sample $x_t = (s_t, a_t, r_t, s_{t+1})$
Initial state $s_0^{(n,k)}$
Given $\psi_{kn}(0) = 1$
for each time slot $t \in \mathcal{T}_n^r$ **do**
  Observe state $s_t^{(n,k)} = \{Z_n(t), l_n(\tau_t), d_{kn}(t)\}$
  Select action:
    $a_t^{(n,k)} = \{M_n(t), l_n(t)\}$
    $= \arg\max_a Q_\theta(s_t, a)$
  Get reward $r_t^{(n,k)}$ and update next state $s_{t+1}^{(n,k)}$
  $\mathcal{D}_{t+1} = \begin{cases} \mathcal{D}_t \cup \{x_t^{(n,k)}\}, & \text{if } |\mathcal{D}_t| < D_m, \\ (\mathcal{D}_t \setminus \{x_{t-D_m}^{(n,k)}\}) \cup \{x_t^{(n,k)}\}, & \text{otherwise.} \end{cases}$
  if $|\mathcal{D}_t| > D_s$ **then** $\theta_{t+1} \leftarrow \theta_t - \alpha \nabla_{\theta_t} \mathcal{L}(\theta_t)$
  if $t = T_{target}$ **then** $\phi_{t+1} \leftarrow \theta_{t+1}$
end for

V. User Scheduling Solution

In this section, we present a stable matching-based scheduling framework that leverages the queue-driven video delivery costs computed in the previous section.

A. Matching-Based Link Scheduling

At the beginning of scheduling round r, each user $n \in \mathcal{N}$ predicts the cumulative delivery cost toward every candidate

RSU $k \in \mathcal{K}_n(t)$, given that the scheduling indicator $\psi_{kn}(t) = 1$ during all $t \in \mathcal{T}_n^r$, as

$$\mathcal{C}^{(n,k)}(r) = \sum_{t \in \mathcal{T}_n^r} f^{(n,k)}(\cdot). \tag{29}$$

These predicted costs are computed by the DQN agent and represent the expected DPP performance over the decision horizon, capturing the effects of queue dynamics and user mobility. Based on these costs, each user n evaluates its candidate RSU set $\mathcal{K}_n(t)$ and arranges the RSUs in ascending order of $\mathcal{C}^{(n,k)}(r)$. The resulting ordered list is used as the user's preference list in the DA procedure.

The algorithm begins by initializing the DA iteration index as $e \leftarrow 1$ and setting the tentative acceptance set for each RSU as $\mathcal{N}_{DA}^{(k)}(r,0) \leftarrow \emptyset$. The initial unmatched-user set is $\mathcal{U} \leftarrow \mathcal{N}$. During iteration e, every unmatched user $n \in \mathcal{U}$ with at least one remaining RSU in its candidate list ($|\mathcal{K}n(t)| > 0$) selects the RSU with the lowest cumulative cost,

$$k_{\text{opt}} = \underset{k \in \mathcal{K}n(t)}{\arg\min}\ \mathcal{C}^{(n,k)}(r), \tag{30}$$

and sends a scheduling proposal $(n, \mathcal{C}^{(n,k\text{opt})}(r))$ to k_{opt}. The selected RSU is then removed from $\mathcal{K}_n(t)$ to prevent duplicate proposals. This step corresponds to the user-side proposal phase in Algorithm 2, where each unmatched user requests service from its currently most preferred RSU.

Upon receiving scheduling proposals, each RSU $k \in \mathcal{K}$ constructs the set of proposers $\mathcal{N}_k^{req}(r,e)$ and combines it with its previously accepted users $\mathcal{N}_{DA}^{(k)}(r,e-1)$ to form

$$\mathcal{U}_k(r,e) = \mathcal{N}_k^{\text{req}}(r,e) \cup \mathcal{N}_{DA}^{(k)}(r,e-1). \tag{31}$$

From this combined set, RSU k tentatively accepts up to J users whose expected cumulative delivery costs are the lowest:

$$\mathcal{N}_{DA}^{(k)}(r,e) = \underset{\substack{\mathcal{S} \subseteq \mathcal{U}_k(r,e) \\ |\mathcal{S}| \le J}}{\arg\min} \sum_{n \in \mathcal{S}} \mathcal{C}^{(n,k)}(r). \tag{32}$$

The remaining users are rejected and form the rejection set

$$\mathcal{R}_k(r,e) = \mathcal{U}_k(r,e) \setminus \mathcal{N}_{DA}^{(k)}(r,e). \tag{33}$$

Intuitively, if a newly proposed user has a lower cost than the highest-cost user currently accepted by RSU k, it replaces that user in $\mathcal{N}_{DA}^{(k)}(r,e)$. Otherwise, it is rejected. After all RSUs complete this process, rejection notifications are sent to users in $\mathcal{R}_k(r,e)$, who then attempt their next preferred RSU in subsequent iterations.

After rejections are processed, the unmatched-user set is updated as

$$\mathcal{U} \leftarrow \mathcal{N} \setminus \bigcup_{k \in \mathcal{K}} \mathcal{N}_{DA}^{(k)}(r,e), \tag{34}$$

which represents all users that are not tentatively accepted by any RSU at iteration e. If no rejections occur in iteration e, i.e., $\sum_{k \in \mathcal{K}} |\mathcal{R}_k(r,e)| = 0$, the procedure has reached convergence and terminates. Otherwise, the iteration index is incremented as $e \leftarrow e+1$, and the process repeats with newly unmatched users proposing again to the next RSU in their preference lists. This iterative update continues for $e = 1, 2, \ldots$ until no unmatched user has an available RSU in its list and no further reassignment occurs across all RSUs.

Algorithm 2 Stable Matching-Based User Scheduling within Round r

Given: $\mathcal{N}$, $\mathcal{K}$, J, $\{\mathcal{K}_n(t)\}_{n \in \mathcal{N}}$, $\{\mathcal{C}^{(n,k)}(r)\}$
Init: $e \leftarrow 1$; $\mathcal{N}_{DA}^{(k)}(r,0) \leftarrow \emptyset$, $\mathcal{U} \leftarrow \mathcal{N}$
while $\exists\, n \in \mathcal{U} :\ |\mathcal{K}_n(t)| > 0$ **do**
#Users propose to their current best RSU
  for each $n \in \mathcal{U}$ with $|\mathcal{K}_n(t)| > 0$ **do**
    $k_{\text{opt}} \leftarrow \arg\min_{k \in \mathcal{K}_n(t)} \mathcal{C}^{(n,k)}(r)$
    send proposal $(n, \mathcal{C}^{(n,k_{\text{opt}})}(r))$ to k_{opt}
    $\mathcal{K}_n(t) \leftarrow \mathcal{K}_n(t) \setminus \{k_{\text{opt}}\}$
  end for
#RSUs tentatively accept up to J lowest-cost proposers
  for each $k \in \mathcal{K}$ **do**
    $\mathcal{N}_k^{\text{req}}(r,e) \leftarrow$ set of proposers to k
    $\mathcal{U}_k(r,e) \leftarrow \mathcal{N}_k^{\text{req}}(r,e) \ \cup\ \mathcal{N}_{DA}^{(k)}(r,e-1)$
    $$\mathcal{N}_{DA}^{(k)}(r,e) = \underset{\substack{\mathcal{S} \subseteq \mathcal{U}_k(r,e) \\ |\mathcal{S}| \le J}}{\arg\min} \sum_{n \in \mathcal{S}} \mathcal{C}^{(n,k)}(r)$$
    $\mathcal{R}_k(r,e) \leftarrow \mathcal{U}_k(r,e) \setminus \mathcal{N}_{DA}^{(k)}(r,e)$
  end for
  if $\sum_{k \in \mathcal{K}} |\mathcal{R}_k(r,e)| = 0$ **then**
    break
  end if
  notify rejections to $\mathcal{R}_k(r,e)$, for all $k \in \mathcal{K}$
#Update unmatched set
  $\mathcal{U} \leftarrow \mathcal{N} \setminus \bigcup_{k \in \mathcal{K}} \mathcal{N}_{DA}^{(k)}(r,e)$
  $e \leftarrow e+1$
end while
Output: with e_{fin}=e, set
$$\psi_{kn}(t) = \begin{cases} 1, & n \in \mathcal{N}_{DA}^{(k)}(r, e_{\text{fin}}), \\ 0, & \textit{otherwise.} \end{cases}$$

Finally, the scheduling indicator is determined as

$$\psi_{kn}(t) = \begin{cases} 1, & n \in \mathcal{N}_{DA}^{(k)}(r, e_{fin}) \\ 0, & \textit{otherwise} \end{cases} \tag{35}$$

where e_{fin} denotes the final iteration index after convergence. According to stable matching theory [51], the above procedure guarantees convergence to a stable one-to-many matching under the capacity constraint J. Because the user preferences are based on long-term, mobility-aware cumulative costs rather than instantaneous channel states, the proposed algorithm ensures stable and scalable scheduling performance even under high-mobility conditions. The complete procedure is summarized in **Algorithm 2**.

B. Computational Complexity

The proposed scheduling algorithm employs a one-to-many stable matching framework with deferred acceptance. At each scheduling round, a subset of users within the observation

region initiates the matching process to associate with nearby RSUs. The total computational cost is determined by two main components: (i) preference list generation using deep reinforcement learning, and (ii) iterative matching execution across users and RSUs.

1) Cumulative DPP Cost Construction: Each of the N users computes its cumulative DPP cost for up to K candidate RSUs by performing DQN inference. The forward inference cost of a DQN is $\mathcal{C}_{\text{DQN}} = \sum_{i=1}^{I} p_{i+1} p_i$, where p_i and p_{i+1} denote the input and output dimensions of the i-th fully connected layer, respectively, and I is the total number of layers. This represents the number of multiply–accumulate operations in one forward pass [52]. Hence, the total cost for preference evaluation in a scheduling round is bounded by $\mathcal{O}(NK\mathcal{C}_{DQN})$.

2) DQN with Learning: Once sufficient experience tuples are collected, the DQN agent begins online training. At each time slot, samples generated from all users are aggregated into a shared replay buffer and used to update the network. The computational cost per update is $\mathcal{C}_{\text{update}} = g\,|\mathcal{D}_t|\,\kappa\,\mathcal{C}_{\text{DQN}}$, where $|\mathcal{D}_t|$ is the mini-batch size, g is the number of gradient steps per update, and $\kappa \in [2, 3]$ accounts for the backward-to-forward cost ratio [53], [54]. Since one update is performed at every slot, the total training complexity per episode of S slots becomes $\mathcal{C}_{\text{learn}} = \mathcal{O}(S\,\mathcal{C}_{\text{update}})$. This formulation captures the continuous slot-wise learning behavior of the DQN agent.

3) Deferred Acceptance Matching: In each matching round, up to N users send proposals to K RSUs. Each RSU maintains a priority queue of size J and performs selection with $\mathcal{O}(\log J)$ complexity per request [55]. The computational cost of one matching iteration is thus $\mathcal{C}_{\text{match}} = \mathcal{O}(NK \log J)$. Then, for an episode of S total slots (i.e., R rounds), the total matching cost is $\sum_{r=1}^{R} e_{\text{fin}}^{(r)} \mathcal{C}_{\text{match}} = \mathcal{O}(R\,\hat{e}_{\text{fin}} NK \log J)$, where $\hat{e}_{\text{fin}}$ is average number of final matching iteration for all scheduling rounds.

4) Overall Complexity: Combining the preference evaluation, DQN learning, and matching processes, the total computational complexity per episode is $\mathcal{O}\big(S\,\mathcal{C}_{\text{update}} + RNK\mathcal{C}_{\text{DQN}} + R\,\hat{e}_{\text{fin}} NK \log J\big)$, which scales linearly with the number of users and RSUs, and remains tractable since J and network depth I are typically small relative to N and K.

VI. Simulation Results

This section presents numerical experiments conducted to evaluate the proposed two-timescale scheduling and delivery framework, highlighting its performance under realistic vehicular mobility and wireless channel dynamics.

A. Simulation Setup

We consider a vehicular video streaming scenario over a road segment of length $d_m = 1$ km and width $d_w = 100$ m, with a total of $N = 100$ vehicles traversing the monitored area. A set of $K = 63$ RSUs are uniformly deployed along both sides of the road at regular intervals of $d_g = 50$ m. The vehicle speed v_n is uniformly distributed in $[v_{\min}, v_{\max}] = [30, 60]$ km/h. New vehicles probabilistically arrive at the beginning of each scheduling round r according to a Poisson process with rate $\lambda = 1$. The maximum number of chunks that can be delivered per slot is $\mu = 9$, i.e., $M_n(t) \in \{0, 1, \ldots, \mu\},\ \forall n$.

Each RSU caches video content encoded at $L = 4$ quality levels, with a uniform caching probability vector $\mathbf{p} = \left[\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4}\right]$. The Lyapunov coefficient is set to $V = 10$ unless otherwise stated. The corresponding chunk sizes for each quality level are given by $S(l_n(t)) = [2.621, 5.073, 10.658, 26.496]$ Kbits. The interference-to-noise ratio (INR) is $\Upsilon = 5$ dB, the path-loss exponent is $\beta = 2$, the available bandwidth is $W = 1$ MHz, and the shadowing variance is $\sigma_w^2 = 4$ dB. The quality measures are given by $\mathcal{P}(l_n(t)) = [34, 36.64, 39.11, 41.64]$ dB. The RSU searching range is $R_s = 50$ m, the slot duration is $t_0 = 0.05$ seconds, the playback consumption rate is $b = 1$ chunk/slot, the fluctuation weight is $\varphi = 0.5$, and the scheduling interval is $T = 25$ slots.

For the DRL-based delivery agent, a pretrained DQN is employed with a learning rate $\eta = 0.0001$, discount factor $\gamma = 0.5$, mini-batch size $|\mathcal{D}_t| = 64$, and replay memory size $|\mathcal{D}| = 20000$. The DQN architecture consists of four fully connected layers. All simulations are implemented in Python 3.8.5 with PyTorch 2.2.1+cu118 on Ubuntu 20.04.6 LTS, executed on an NVIDIA GeForce RTX 3090 GPU.

B. Baselines

To evaluate the performance of the proposed DQN-based matching and content delivery framework, several benchmark schemes are implemented under the same network conditions and parameter settings.

*a) **Distance-Based Matching (DBM)**:* Unlike the proposed scheme, where the DQN agent predicts the future cumulative DPP $\mathcal{C}^{(n,k)}(r)$ to generate preference scores for matching, DBM employs the geographical distance $d_{kn}(t)$ between each vehicle and RSU as the matching score. The same matching algorithm as the proposed method is applied, while the content delivery process utilizes a DQN agent trained in the DBM scheduling environment to ensure a fair comparison. In this scheme, scheduling decisions are made to minimize the instantaneous sum of vehicle–RSU distances at time t, without considering any future variations in vehicle mobility or RSU association dynamics.

*b) **Hierarchical Reinforcement Learning (HRL)**:* This baseline is designed with reference to [56], where video delivery and RSU scheduling are independently controlled by two separate DQN agents. The video delivery agent operates in the same manner as in the proposed method. In contrast, the scheduling agent observes the state $s_{sch}(t) = \{Z_n(t), \tilde{l}_n(t), d_{kn}(t)\}$ for up to K_{sch} RSUs within the communication range of vehicle n, and generates the action-value list $\mathcal{Q}^{(n,k)}\theta_{sch}(s_{sch}(t), a_{sch}(t))$. According to [56], the scheduling DQN is trained to maximize the cumulative reward obtained from the delivery agent, i.e., $\sum_{t \in \mathcal{T}_n^r} r_{del}^{(n,k)}(t)$, such that RSUs with higher cumulative delivery rewards yield higher action values $Q_{\theta_{sch}}(s_{sch}^{(n,k)}(t), a_{sch}^{(n,k)}(t))$. In this work, the delivery reward is defined as $r_{del}^{(n,k)}(t) = f^{(n,k)}(\cdot)$. The resulting action-value list $\mathcal{Q}_{\theta_{sch}}(s_{sch}^{(n,k)}(t), a_{sch}^{(n,k)}(t))$ for all RSUs k within the communication range is used as the

preference list for matching, to which **Algorithm 2** is applied. Therefore, unlike the proposed method that predicts future cumulative costs using the delivery DQN agent, HRL performs scheduling based on the Q-value predictions of the scheduling DQN agent, which is hierarchically linked to the delivery agent through the reward structure.

*c) **Advantage Actor-Critic (A2C):*** This baseline replaces the DQN agent in the proposed framework with an A2C agent [57], while all other configurations, reward structures, and matching mechanisms remain identical.

*d) **Suboptimal (24):*** This baseline assumes perfect knowledge of the instantaneous channel capacity $C_{kn}(t)$ during content delivery. With this information, the agent solves problem (24) by performing a full search over all actions that satisfy constraint (11), thereby determining the optimal delivery vector $\mathbf{l}^r n$ and $\mathbf{M}^r n$. For each time slot t, the delivery agent selects the action that minimizes the DPP among all feasible actions, representing an idealized suboptimal delivery policy. The scheduling process follows the proposed framework, where the DQN agent predicts the future cumulative DPP for matching decisions.

*e) **Suboptimal (25):*** In this baseline, the scheduling process is assumed to operate optimally. The scheduler is endowed with perfect foresight of the future channel capacities $C_{kn}(t)$ for all $t \in \mathcal{T}_n^r$ when predicting $\mathcal{C}^{(n,k)}(r)$. For each candidate RSU, the cumulative DPP is evaluated by selecting the optimal action—specifically, the one that minimizes the DPP over the entire action space. Scheduling decisions are then made based on these ideal cumulative DPP values, corresponding to the optimal solution of subproblem (25). To fully reflect the optimal scheduling assumption, the per-RSU resource constraint J is relaxed, allowing each RSU to simultaneously serve all associated vehicles within the round.

C. Comparison Work

This section compares the proposed method with the baseline schemes in terms of buffering rate, average chunk size, and extreme quality fluctuation rate under various conditions. The *buffering rate* represents the ratio of the average number of buffering occurrences per vehicle during the observation period. The *average chunk size*, equivalent to the average bitrate, indicates the mean file size per video chunk played by each vehicle. The *extreme quality fluctuation rate* denotes the ratio of chunks in which the video quality level changes by two or more consecutive levels. These three metrics are jointly used to evaluate the user Quality of Experience (QoE).

a) Effect of Caching Distribution: As illustrated in Fig. **??**, the performance is evaluated under different caching probability cases defined as follows:

- **Case 1:** $\mathbf{p} = [6/12, 3/12, 2/12, 1/12]$, where RSUs are more likely to cache videos at lower quality levels.
- **Case 2:** $\mathbf{p} = [1/4, 1/4, 1/4, 1/4]$, where all quality levels are cached with equal probability.
- **Case 3:** $\mathbf{p} = [1/12, 2/12, 3/12, 6/12]$, where RSUs are more likely to cache higher-quality video contents.

From Fig. **??**, it can be observed that as more RSUs cache higher-quality contents, all methods achieve higher delivery quality while exhibiting a lower buffering rate. This indicates that vehicles are less likely to associate with distant RSUs to obtain higher-quality videos. Furthermore, all methods except **Suboptimal (24)** show no occurrence of quality fluctuations. In **Suboptimal (24)**, the fluctuation rate is highest in **Case 3** because a greater number of high-quality cached contents increases the variability of selectable quality levels $l_n(t)$, resulting in more frequent quality transitions. In addition, since the weighting factor $\varphi = 0.5$ assigns relatively less importance to queue stability compared to bitrate in the objective function, the **Suboptimal (24)** case yields the highest fluctuation rate. However, the scale of fluctuation remains extremely small, indicating that such fluctuations rarely occur in practice. Across all performance metrics, the proposed method achieves a lower buffering rate and higher bitrate compared to the baseline schemes, excluding the idealized suboptimal methods **Suboptimal (24)** and **Suboptimal (25)**, which assume perfect channel knowledge. Notably, even in **Case 1**, where RSUs predominantly cache lower-quality contents, the proposed method achieves an average chunk size exceeding 5.073 Kbits (the file size corresponding to quality level 2).

b) Effect of Transmit SNR: As shown in Fig. **??**, all baselines exhibit a similar trend in buffering rate, which decreases as the transmit SNR increases. A comparable trend is also observed in the average bitrate, where both **Suboptimal (25)** and the proposed method consistently achieve higher bitrates as SNR increases. This improvement arises because higher SNR enables vehicles to reliably receive services from more distant RSUs, facilitating dynamic video delivery; thus, the predictive scheduling based on future cumulative DPP becomes increasingly effective. The quality fluctuation rate shows a similar pattern to Fig. **??**, where higher SNR results in slightly increased fluctuation due to the enhanced adaptability of dynamic delivery decisions.

c) Effect of System Parameter V: Fig. **??** illustrates how the system parameter V influences the trade-off among buffering rate, average bitrate, and quality fluctuation. As V controls the weighting between queue stability and bitrate in the Lyapunov-based objective, it enables flexible performance tuning depending on system priorities. Specifically, a smaller V results in a lower buffering rate, a trend consistently observed across all baselines. For the average bitrate, however, the rate of increase varies among methods. The proposed method consistently achieves higher bitrates than other baselines (except for the idealized **Suboptimal (25)**) while maintaining a lower buffering rate than all other baselines except the suboptimal schemes. This demonstrates that the proposed method can flexibly control the performance trade-off while achieving superior overall QoE. Moreover, the quality fluctuation rate remains nearly constant as V increases, indicating that larger V values do not significantly promote dynamic delivery behavior.

VII. Evaluation with Real-World Data

-DASH dataset을 사용, CARLA, GStreamer simulator tool을 사용함을 설명

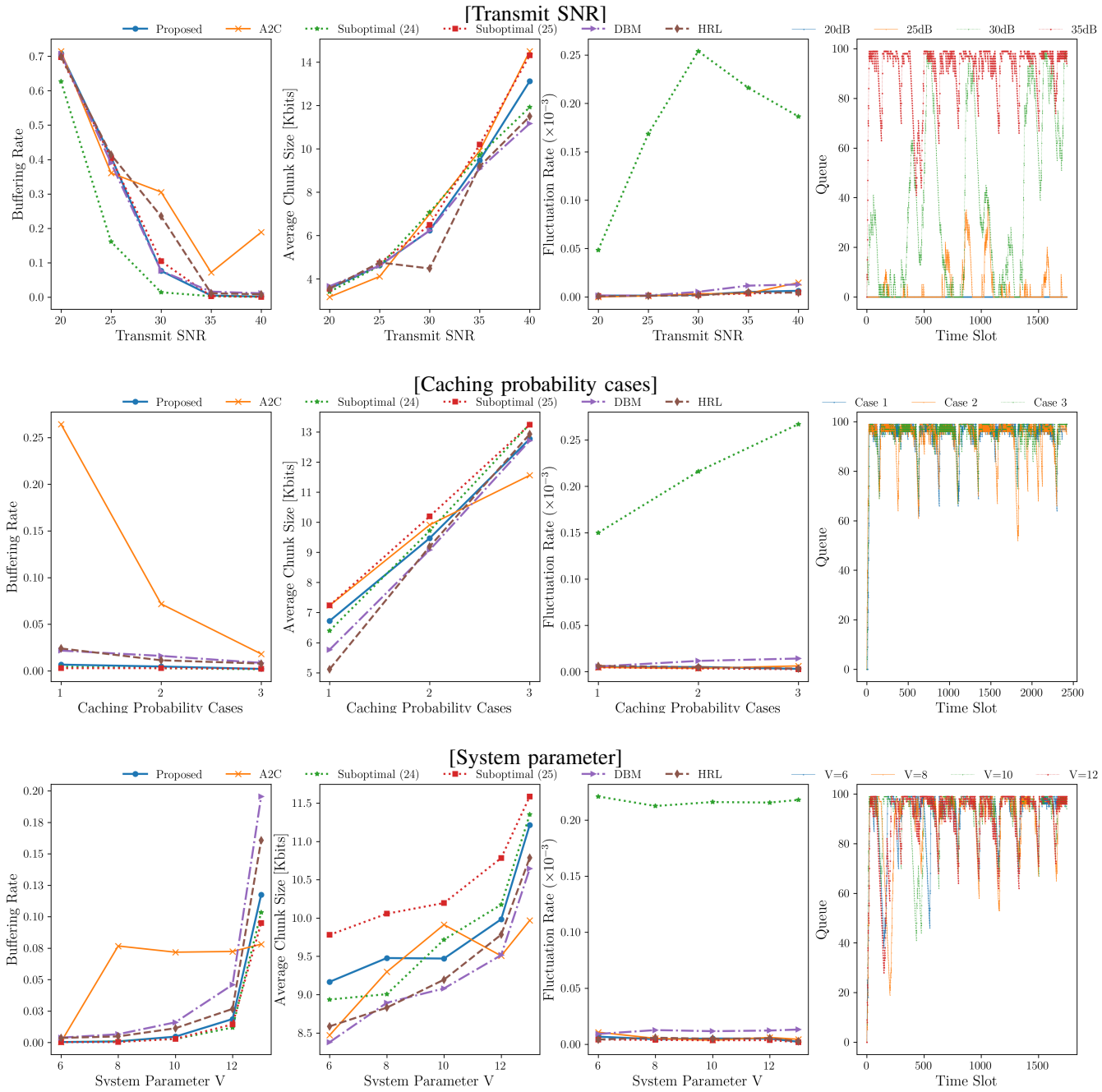

Fig. 2: Performance Evaluations

A. Integration with the Proposed Framework

simulator상에 우리의 알고리즘을 어떻게 적용했는지 detail, setup을 설명

B. Performance Evaluation on Real Environment

simulator 상에서의 실험 결과

C. Discussion

VI에서의 결과와 비교하여 real-world에서의 차이점 discussion

VIII. Conclusion

Acknowledgment

The authors would like to thank...

References

[1] X. Huang, L. He, L. Wang, and F. Li, "Towards 5g: Joint optimization of video segment caching, transcoding and resource allocation for adaptive video streaming in a multi-access edge computing network," *IEEE Transactions on Vehicular Technology*, vol. 70, no. 10, pp. 10 909–10 924, 2021.

[2] D. Naseh, A. Bozorgchenani, and D. Tarchi, "Deep reinforcement learning for edge-dash-based dynamic video streaming," in *2025 IEEE Wireless Communications and Networking Conference (WCNC)*. IEEE, 2025, pp. 1–6.

[3] B. Feng, C. Feng, G. Min, and T. Q. Quek, "Two-timescale adaptive live video streaming transmission mechanism for vehicular networks," *IEEE Transactions on Vehicular Technology*, 2024.

[4] M. Choi, A. No, M. Ji, and J. Kim, "Markov decision policies for dynamic video delivery in wireless caching networks," *IEEE Transactions on Wireless Communications*, vol. 18, no. 12, pp. 5705–5718, 2019.

[5] F. Bouali, K. Moessner, and M. Fitch, "Multi-timescale qoe provisioning for adaptive video streaming in heterogeneous deployments," *IEEE Transactions on Vehicular Technology*, vol. 70, no. 2, pp. 1894–1909, 2021.

[6] H. Zhan, L. Fan, C. Li, X. Lei, and F. Li, "Cloud-edge learning for adaptive video streaming in b5g internet-of-thing systems," *IEEE Internet of Things Journal*, 2024.

[7] J. Yang, P. Si, Z. Wang, X. Jiang, and L. Hanzo, "Dynamic resource allocation and layer selection for scalable video streaming in femtocell networks: A twin-time-scale approach," *IEEE Transactions on Communications*, vol. 66, no. 8, pp. 3455–3470, 2018.

[8] W. Yang, X. Chi, L. Zhao, and R. Qi, "Predictive two-timescale resource allocation for vod services in fast moving scenarios," *IEEE Transactions on Vehicular Technology*, vol. 70, no. 10, pp. 10 002–10 017, 2021.

[9] B. Feng, C. Feng, G. Min, and T. Q. S. Quek, "Two-timescale adaptive live video streaming transmission mechanism for vehicular networks," *IEEE Transactions on Vehicular Technology*, vol. 74, no. 4, pp. 6823–6828, 2025.

[10] S. Wang, J. Yang, and S. Bi, "Adaptive video streaming in multi-tier computing networks: Joint edge transcoding and client enhancement," *IEEE Transactions on Mobile Computing*, vol. 23, no. 4, pp. 2657–2670, 2023.

[11] W. Li, J. Huang, S. Wang, C. Wu, S. Liu, and J. Wang, "An apprenticeship learning approach for adaptive video streaming based on chunk quality and user preference," *IEEE Transactions on Multimedia*, vol. 25, pp. 2488–2502, 2022.

[12] S. Wang, S. Bi, and Y.-J. A. Zhang, "Deep reinforcement learning with communication transformer for adaptive live streaming in wireless edge networks," *IEEE Journal on Selected Areas in Communications*, vol. 40, no. 1, pp. 308–322, 2021.

[13] B. Hou, S. Yang, F. A. Kuipers, L. Jiao, and X. Fu, "Eavs: Edge-assisted adaptive video streaming with fine-grained serverless pipelines," in *IEEE INFOCOM 2023-IEEE Conference on Computer Communications*. IEEE, 2023, pp. 1–10.

[14] I. Bartolec, I. Orsolic, and L. Skorin-Kapov, "Impact of user playback interactions on in-network estimation of video streaming performance," *IEEE transactions on network and service management*, vol. 19, no. 3, pp. 3547–3561, 2022.

[15] T. Huang, R.-X. Zhang, C. Wu, and L. Sun, "Optimizing adaptive video streaming with human feedback," in *Proceedings of the 31st ACM International Conference on Multimedia*, 2023, pp. 1707–1718.

[16] L. A. binti Burhanuddin, X. Liu, Y. Deng, U. Challita, and A. Zahemszky, "Qoe optimization for live video streaming in uav-to-uav communications via deep reinforcement learning," *IEEE Transactions on Vehicular Technology*, vol. 71, no. 5, pp. 5358–5370, 2022.

[17] N. Kan, J. Zou, C. Li, W. Dai, and H. Xiong, "Rapt360: Reinforcement learning-based rate adaptation for 360-degree video streaming with adaptive prediction and tiling," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 32, no. 3, pp. 1607–1623, 2021.

[18] J. Liu, Z. Liu, J. Huang, W. Jiang, and J. Wang, "A buffer-based adaptive bitrate approach in wireless networks with iterative correction," *IEEE Wireless Communications Letters*, vol. 11, no. 8, pp. 1644–1648, 2022.

[19] J. Ye, S. Qin, Q. Xiao, W. Jiang, X. Tang, and X. Li, "Adaptive bitrate algorithms via deep reinforcement learning with digital twins assisted trajectory," *IEEE Transactions on Network Science and Engineering*, 2024.

[20] J. Zhang, H. Wu, X. Tao, and X. Zhang, "Adaptive bitrate video streaming in non-orthogonal multiple access networks," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 4, pp. 3980–3993, 2020.

[21] X. Zuo, J. Yang, M. Wang, and Y. Cui, "Adaptive bitrate with user-level qoe preference for video streaming," in *IEEE INFOCOM 2022-IEEE Conference on Computer Communications*. IEEE, 2022, pp. 1279–1288.

[22] M. Alahmadi, P. Pocta, and H. Melvin, "An adaptive bitrate switching algorithm for speech applications in context of webrtc," *ACM Transactions on Multimedia Computing, Communications, and Applications (TOMM)*, vol. 17, no. 4, pp. 1–21, 2021.

[23] A. Bentaleb, M. Lim, M. N. Akcay, A. C. Begen, and R. Zimmermann, "Bitrate adaptation and guidance with meta reinforcement learning," *IEEE Transactions on Mobile Computing*, 2024.

[24] W. Li, X. Li, Y. Xu, Y. Yang, and S. Lu, "Metaabr: A meta-learning approach on adaptative bitrate selection for video streaming," *IEEE Transactions on Mobile Computing*, vol. 23, no. 3, pp. 2422–2437, 2023.

[25] T. Karagkioules, G. S. Paschos, N. Liakopoulos, A. Fiandrotti, D. Tsilimantos, and M. Cagnazzo, "Online learning for adaptive video streaming in mobile networks," *ACM Transactions on Multimedia Computing, Communications, and Applications (TOMM)*, vol. 18, no. 1, pp. 1–22, 2022.

[26] A.-T. Tran, N.-N. Dao, and S. Cho, "Bitrate adaptation for video streaming services in edge caching systems," *IEEE Access*, vol. 8, pp. 135 844–135 852, 2020.

[27] N.-P. Tran, T. P. Truong, Q. T. Do, N.-N. Dao, and S. Cho, "Joint wireless resource allocation and bitrate adaptation for qoe improvement in irs-aided rsma-enabled iomt streaming systems," *Internet of Things*, vol. 25, p. 101145, 2024.

[28] Z. Wang, G. Liu, D. Zhang, X. Hua, L. Xu, P. Gao, and T. Jiang, "Edge-assisted massive video delivery over cell-free massive mimo," *IEEE Transactions on Multimedia*, vol. 25, pp. 8566–8579, 2023.

[29] Q. Lan, B. Lv, R. Wang, K. Huang, and Y. Gong, "Adaptive video streaming for massive mimo networks via approximate mdp and reinforcement learning," *IEEE Transactions on Wireless Communications*, vol. 19, no. 9, pp. 5716–5731, 2020.

[30] M. Tang and V. W. Wong, "Online bitrate selection for viewport adaptive 360-degree video streaming," *IEEE Transactions on Mobile Computing*, vol. 21, no. 7, pp. 2506–2517, 2020.

[31] W. Li, J. Huang, W. Lyu, B. Guo, W. Jiang, and J. Wang, "Rav: Learning-based adaptive streaming to coordinate the audio and video bitrate selections," *IEEE Transactions on Multimedia*, vol. 25, pp. 5662–5675, 2022.

[32] K. Shen, D. Zhang, Z. Zhu, L. Zhang, F. Wang, and D. Wang, "Sja: Server-driven joint adaptation of loss and bitrate for multi-party realtime video streaming," in *IEEE INFOCOM 2023-IEEE Conference on Computer Communications*. IEEE, 2023, pp. 1–10.

[33] C. Feres and Z. Ding, "An unsupervised learning paradigm for user scheduling in large scale multi-antenna systems," *IEEE Transactions on Wireless Communications*, vol. 22, no. 5, pp. 2932–2945, 2022.

[34] D. Liu, G. Zhu, J. Zhang, and K. Huang, "Data-importance aware user scheduling for communication-efficient edge machine learning," *IEEE Transactions on Cognitive Communications and Networking*, vol. 7, no. 1, pp. 265–278, 2020.

[35] Y. Li, Y. Wu, Y. Song, L. Qian, and W. Jia, "Dynamic user-scheduling and power allocation for swipt aided federated learning: A deep learning approach," *IEEE Transactions on Mobile Computing*, vol. 22, no. 12, pp. 6956–6969, 2022.

[36] D. Peng, A. Bandi, Y. Li, S. Chatzinotas, and B. Ottersten, "Hybrid beamforming, user scheduling, and resource allocation for integrated terrestrial-satellite communication," *IEEE Transactions on Vehicular Technology*, vol. 70, no. 9, pp. 8868–8882, 2021.

[37] S. He, J. Yuan, Z. An, W. Huang, Y. Huang, and Y. Zhang, "Joint user scheduling and beamforming design for multiuser miso downlink systems," *IEEE Transactions on Wireless Communications*, vol. 22, no. 5, pp. 2975–2988, 2022.

[38] C. Xu, Y. Jia, S. He, Y. Huang, and D. Niyato, "Joint user scheduling, base station clustering, and beamforming design based on deep unfolding technique," *IEEE Transactions on Communications*, vol. 71, no. 10, pp. 5831–5845, 2023.

[39] Z. Zhang, T. Jiang, and W. Yu, "Learning based user scheduling in reconfigurable intelligent surface assisted multiuser downlink," *IEEE Journal of Selected Topics in Signal Processing*, vol. 16, no. 5, pp. 1026–1039, 2022.

[40] X. Zhao, Y.-J. A. Zhang, M. Wang, X. Chen, and Y. Li, "Online multi-user scheduling for xr transmissions with hard-latency constraint: Performance analysis and practical design," *IEEE Transactions on Communications*, 2024.

[41] Z. Ning, P. Dong, X. Wang, X. Hu, J. Liu, L. Guo, B. Hu, R. Y. Kwok, and V. C. Leung, "Partial computation offloading and adaptive task scheduling for 5g-enabled vehicular networks," *IEEE Transactions on Mobile Computing*, vol. 21, no. 4, pp. 1319–1333, 2020.

[42] H. Zhang, L. Feng, X. Liu, K. Long, and G. K. Karagiannidis, "User scheduling and task offloading in multi-tier computing 6g vehicular network," *IEEE Journal on Selected Areas in Communications*, vol. 41, no. 2, pp. 446–456, 2022.

[43] Z. Si, G. Chuai, K. Zhang, W. Gao, X. Chen, and X. Liu, "Backhaul capacity-limited joint user association and power allocation scheme in ultra-dense millimeter-wave networks," *Entropy*, vol. 25, no. 3, p. 409, 2023.

[44] S. Kishk, N. Almofari, and F. W. Zaki, "Distributed resource allocation in d2d communication networks with energy harvesting relays using stable matching," *Ad Hoc Networks*, vol. 61, pp. 114–123, 2017.
[45] T. LeAnh, N. H. Tran, W. Saad, L. B. Le, D. Niyato, T. M. Ho, and C. S. Hong, "Matching theory for distributed user association and resource allocation in cognitive femtocell networks," *IEEE Transactions on Vehicular Technology*, vol. 66, no. 9, pp. 8413–8428, 2017.
[46] T. Hößler, P. Schulz, E. A. Jorswieck, M. Simsek, and G. P. Fettweis, "Stable matching for wireless urllc in multi-cellular, multi-user systems," *IEEE Transactions on Communications*, vol. 68, no. 8, pp. 5228–5241, 2020.
[47] M. W. Baidas, Z. Bahbahani, and E. Alsusa, "User association and channel assignment in downlink multi-cell noma networks: A matching-theoretic approach," *EURASIP Journal on Wireless Communications and Networking*, vol. 2019, no. 1, p. 220, 2019.
[48] M. Choi, J. Kim, and J. Moon, "Wireless video caching and dynamic streaming under differentiated quality requirements," *IEEE Journal on Selected Areas in Communications*, vol. 36, no. 6, pp. 1245–1257, 2018.
[49] M. Choi, T. Xiang, and J. Kim, "Intelligent caching for seamless high-quality streaming in vehicular networks: A multi-agent reinforcement learning approach," *IEEE Transactions on Intelligent Vehicles*, vol. 9, no. 2, pp. 3672–3686, 2024.
[50] M. Neely, *Stochastic network optimization with application to communication and queueing systems*. Morgan & Claypool Publishers, 2010.
[51] D. Gale and L. S. Shapley, "College admissions and the stability of marriage," *The American mathematical monthly*, vol. 69, no. 1, pp. 9–15, 1962.
[52] P. Freireetal, "Computationalcomplexityoptimizationofneuralnetwork-based equalizers in digital signal processing: A comprehensive approach," *J. Lightw. Technol*, vol. 42, no. 12, pp. 4177–4201, 2024.
[53] A. H. Jiang, D. L.-K. Wong, G. Zhou, D. G. Andersen, J. Dean, G. R. Ganger, G. Joshi, M. Kaminksy, M. Kozuch, Z. C. Lipton *et al.*, "Accelerating deep learning by focusing on the biggest losers," *arXiv preprint arXiv:1910.00762*, 2019.
[54] C. T. Fokam, K. K. Nazeer, L. König, D. Kappel, and A. Subramoney, "Asynchronous stochastic gradient descent with decoupled backpropagation and layer-wise updates," *arXiv preprint arXiv:2410.05985*, 2024.
[55] C. Thomas H, E. Charles, R. Ronald L, S. Clifford *et al.*, "Introduction to algorithms third edition," 2009.
[56] Y. Kim, T. Xiang, Y. Kim, and M. Choi, "Hierarchical reinforcement learning-based delay-sensitive video delivery in vehicular networks," *한국통신학회논문지*, vol. 49, no. 8, pp. 1110–1117, 2024.
[57] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," in *International conference on machine learning*. PmLR, 2016, pp. 1928–1937.

Yunoh Kim received the B.S. degree in Electronic Engineering in 2023 and the M.S. degree in Electronics and Information Convergence Engineering in 2025, both from Kyung Hee University, Yongin, South Korea. He is currently a Ph.D. student at Kyung Hee University, Yongin, South Korea. His research interests include distributed intelligence systems, federated learning, and reinforcement learning.

Minseok Choi received the B.S., M.S., and Ph.D. degrees from the School of Electrical Engineering, Korea Advanced Institute of Science and Technology, Daejeon, South Korea, in 2011, 2013, and 2018, respectively. He is an Assistant Professor of electronic engineering with Kyung Hee University, Yongin, South Korea. He was a Visiting Postdoctoral Researcher in electrical and computer engineering with the University of Southern California, Los Angeles, CA, USA. His research interests include wireless caching networks, federated learning, reinforcement learning, and stochastic network optimization. He was the recipient of the IEEE Communications Society (ComSoc) Multimedia Communications Technical Committee (MMTC) Best Paper Award, 2022.